DFD

Data Flow Diagram (DFD) is a description of the flow of information from the input process to the output of a system. DFD is used because many workflows are more difficult to describe with word descriptions and will be more effective when using diagrams.

DFD Symbols

There are four basic symbols that are used to represent a data-flow diagram.

1. Process

A process receives input data and produces output with a different content or form. Processes can be as simple as collecting input data and saving in the database, or it can be complex as producing a report containing monthly sales of all retail stores in the northwest region.

Every process has a name that identifies the function it performs.

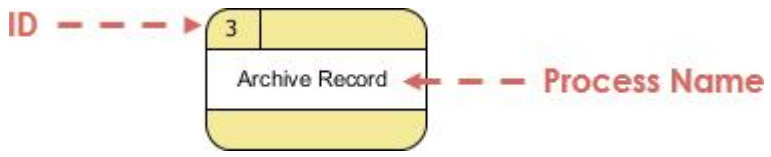
The name consists of a verb, followed by a singular noun.

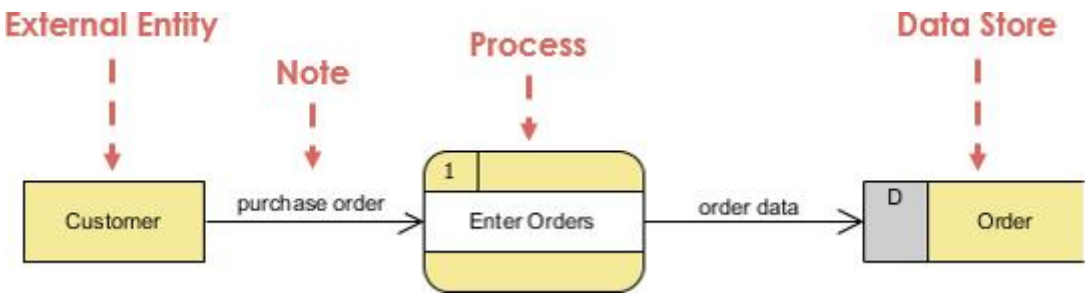
Example:

- ✓ Apply Payment
- ✓ Calculate Commission
- ✓ Verify Order

Notation

- ✓ A rounded rectangle represents a process
- ✓ Processes are given IDs for easy referencing





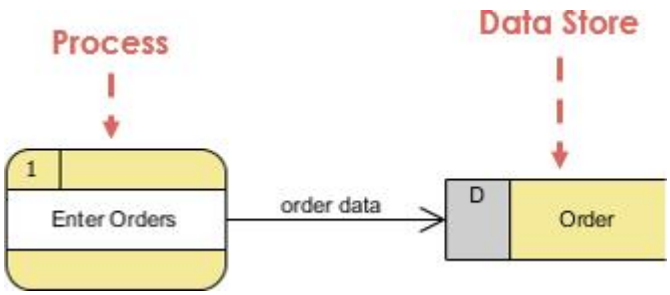
2. Data Flow

A data-flow is a path for data to move from one part of the information system to another. A data-flow may represent a single data element such the Customer ID or it can represent a set of data element (or a data structure).

Example:

Customer_info (LastName, FirstName, SS#, Tel #, etc.)

Order_data(OrderId, Item#, OrderDate, CustomerID, etc.).



Notation

- ✓ Straight lines with incoming arrows are input data flow
- ✓ Straight lines with outgoing arrows are output data flows

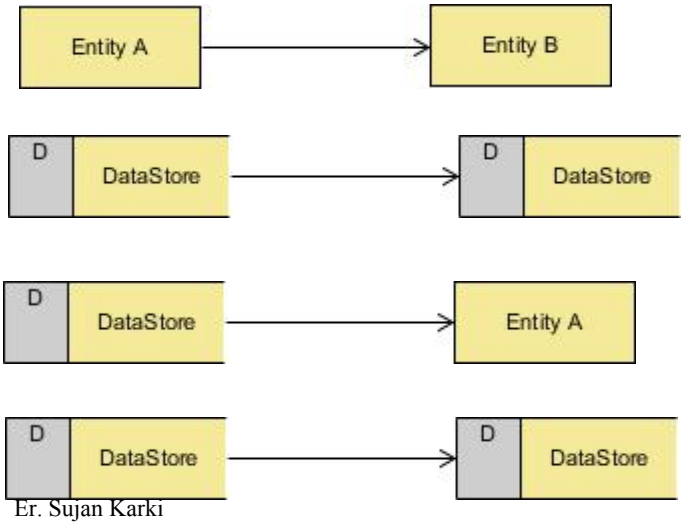
Note that:

Because every process changes data from one form into another, at least one data-flow must enter and one data-flow must exit each process symbol.

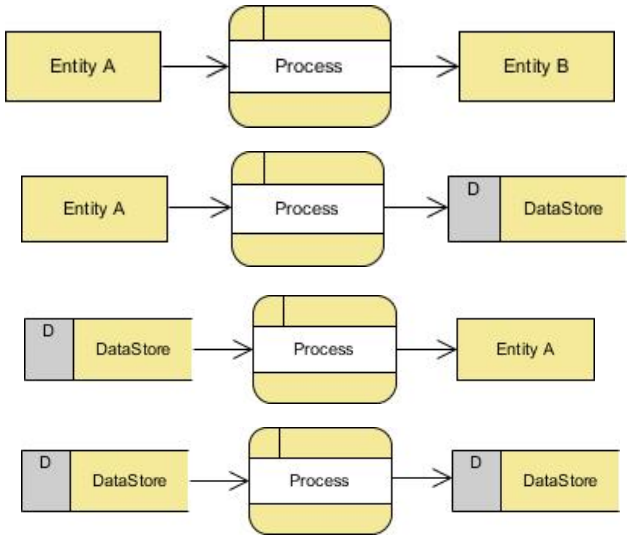
Rule of Data Flow

One of the rule for developing DFD is that all flow must begin with and end at a processing step. This is quite logical, because data can't transform on its own with being process. By using the thumb rule, it is quite easily to identify the illegal data flows and correct them in a DFD.

WRONG



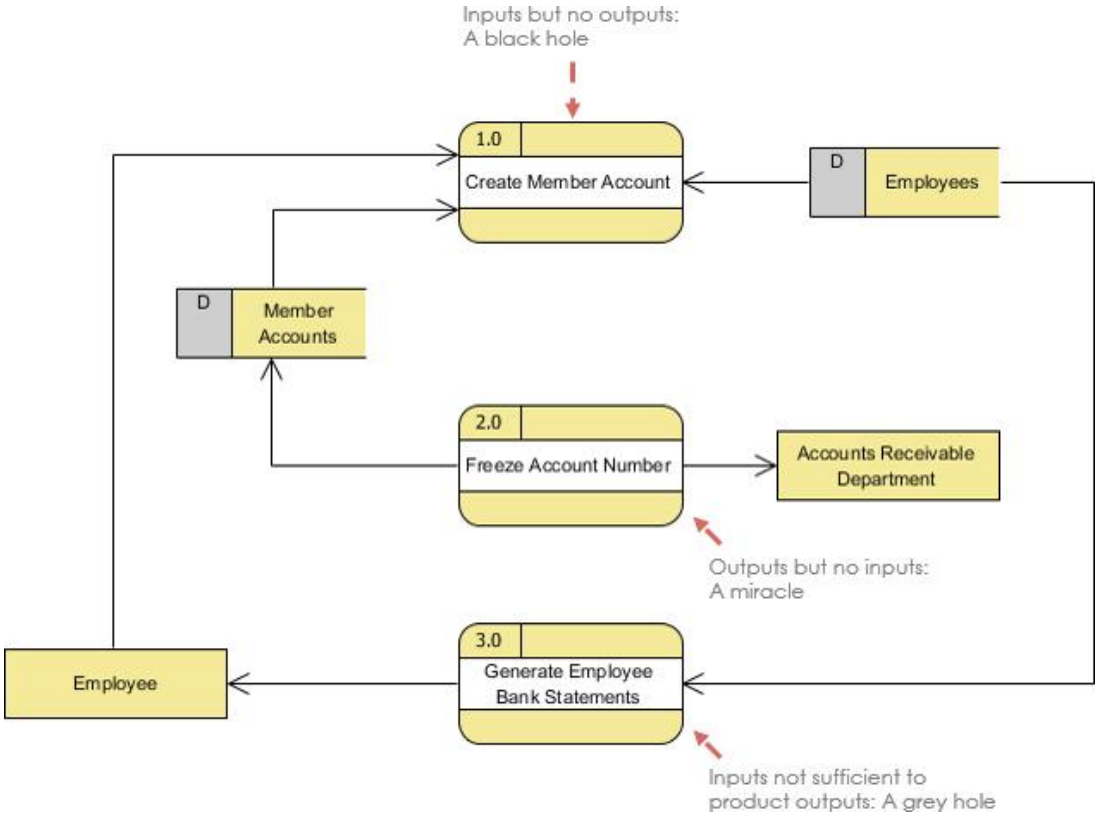
RIGHT



- ✓ An entity cannot provide data to another entity without some processing occurred.
- ✓ Data cannot move directly from an entity to a data store without being processed.
- ✓ Data cannot move directly from a data store without being processed.
- ✓ Data cannot move directly from one data store to another without being processed.

Other frequently-made mistakes in DFD

Black holes - A processing step may have input flows but no output flows.
Miracles - A processing step may have output flows but no input flows.
Grey holes - A processing step may have outputs that are greater than the sum of its inputs.



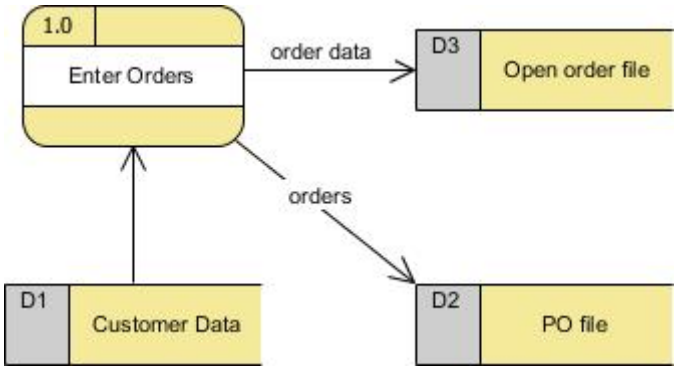
3. Data Store

A data store or data repository is used in a data-flow diagram to represent a situation when the system must retain data because one or more processes need to use the stored data in a later time.

Notation

Data can be written into a data store, represented by a flow connector flowing from the 'writer' of the data to the data store.

Data can be read from a data store, represented by a flow connector flowing from the data store to the 'reader' of the data.
Examples of data stores include Inventory, Accounts Receivable, Orders, and Daily Payments.



Note that:

- ✓ A data store must be connected to a process with a data-flow.
- ✓ Each data store must have at least one input data-flow and at least one output data-flow (even if the output data-flow is a control or confirmation message).

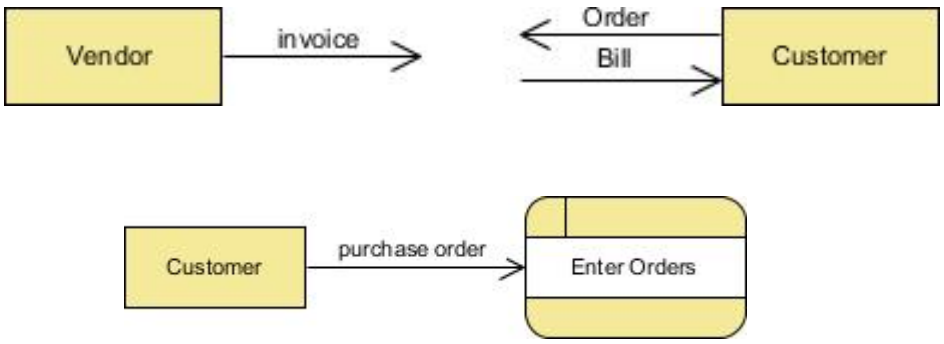
4. External Entity

An external entity is a person, department, outside organization, or other information system that provides data to the system or receives outputs from the system. External entities are components outside of the boundaries of the information systems. They represent how the information system interacts with the outside world.

- ✓ A rectangle represents an external entity
- ✓ They either supply data or receive data
- ✓ They do not process data
- ✓ An external entity may be:
  - ■ Source only: sends data to the system only
  - ■ Sink only: receives data from the system only
  - ■ Both source and sink: sends data to and receives data from the system

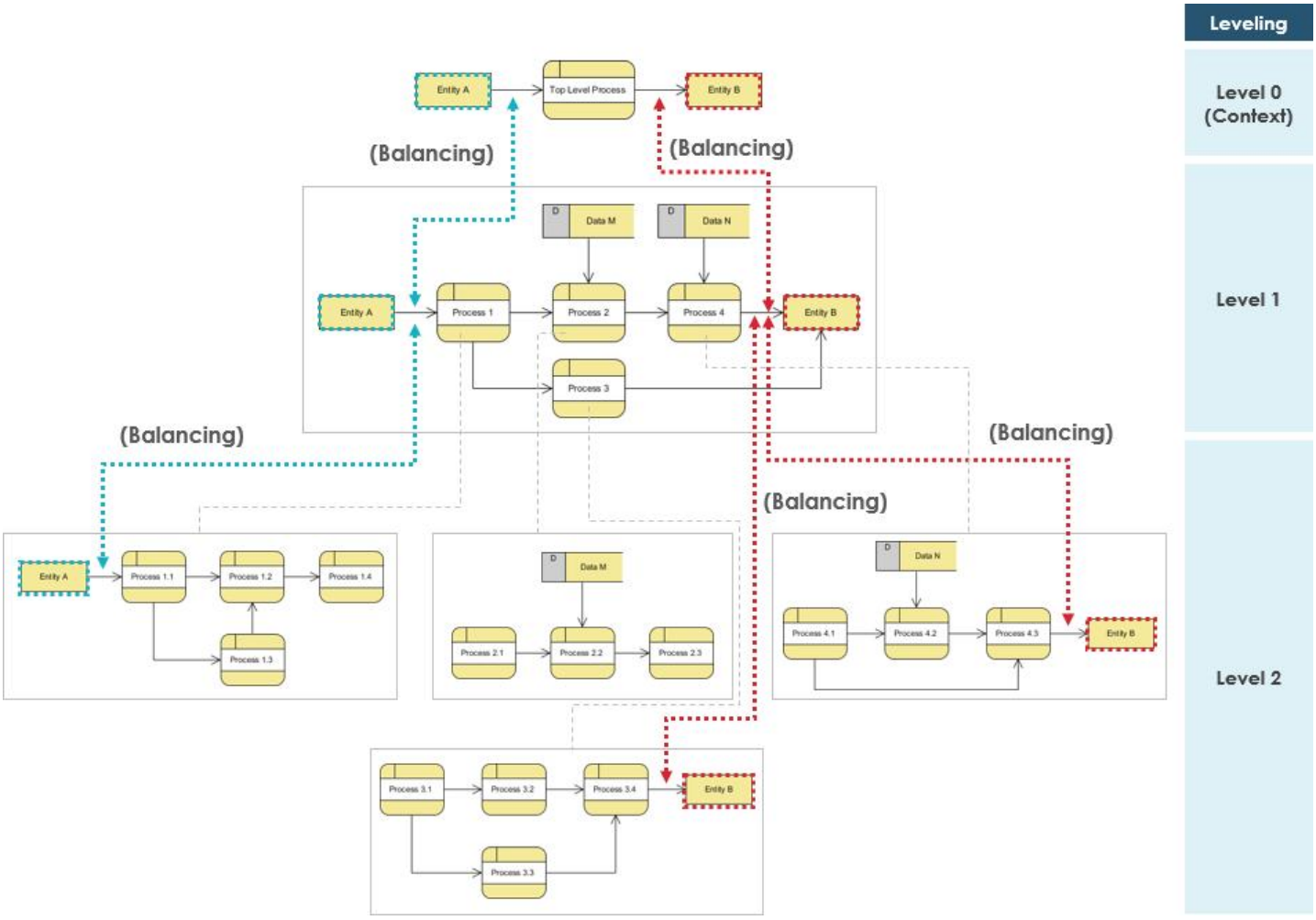
Notation

- ✓ A customer submitting an order and then receive a bill from the system
- ✓ A vendor issue an invoice



Note that:

- ✓ External entities also are called terminators because they are data origins or final destinations.
- ✓ An external entity must be connected to a process through a data-flow.



Guideline for Developing Data-Flow Diagram

Unique Name for Levels

- ✓ Use unique names within each set of symbols.
  For example, there can be only one entity CUSTOMER in all levels of the data-flow diagrams; or here can be only one process named CALCULATE OVERTIME among all levels of data-flow diagrams.

No Cross Line in DFD

- ✓ One way to achieve this is to restrict the number of processes in a data-flow diagram.

Right Complexity for Human Mind: 7 ± 2 Symbols

The idea 7 ± 2 means a person can comfortably understand around:

- ✓ Minimum: 5 symbols
- ✓ Ideal: around 7 symbols
- ✓ Maximum: 9 symbols

In a lower-level DFD, especially Level 1 or Level 2, try to keep the number of processes between 5 and 9.

Why? When too many processes are placed in one diagram:

- ✓ The diagram becomes crowded.
- ✓ Arrows cross each other.
- ✓ It becomes difficult to identify which data goes where.
- ✓ Students and system users may misunderstand the system.

So, a good DFD should not try to show every tiny detail on one page.

Numbering Convention

- ✓ Use a unique reference number for each process symbol.
- ✓ Other process numbers are in the hierarchy of:
  (1, 2, 3,...)
  (1.1, 1.2, 1.3, ..., 2.1, 2.2, 2.3,...)
  (1.1.1, 1.1.2, 1.1.3,...)

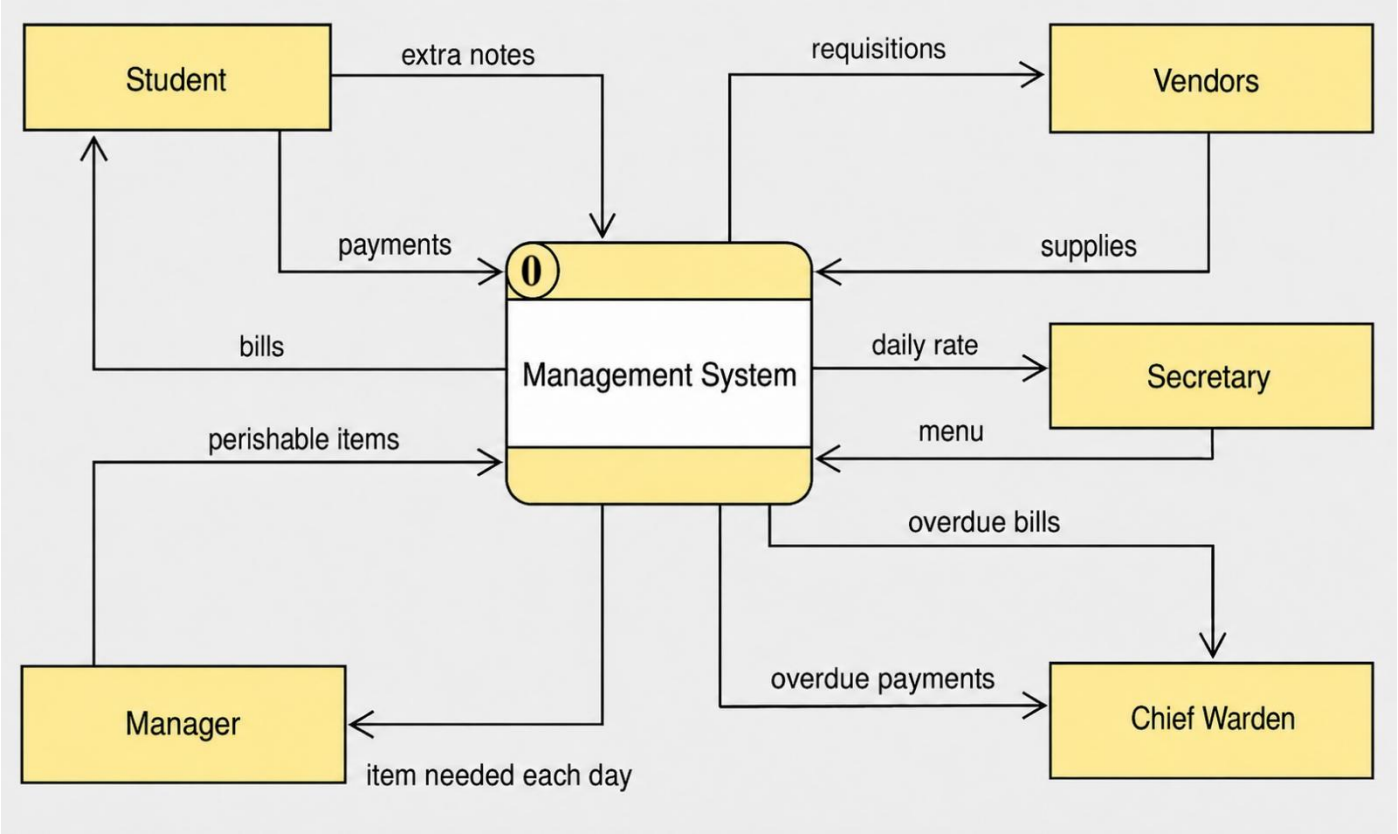
Context Diagram - Level 0 (Highest Level DFD)

A Context-Level Diagram gives an overall view of the entire system. It contains only one process, which represents the complete information system.

The single process is connected with all the important external entities that interact with the system.

- ✓ The context diagram must fit in one page. So do not add many internal processes, databases, or detailed steps in it.
- ✓ The process name in the context diagram should be the name of the information system.

  For example, Grading System, Order Processing System, Registration System.
- ✓ The context level diagram gets the number 0 (level zero).
- ✓ The single process in the Context Diagram cannot show all internal activities of the system.
- ✓ Therefore, the process is broken down into major processes in the next-level DFD. This is called exploding the process.



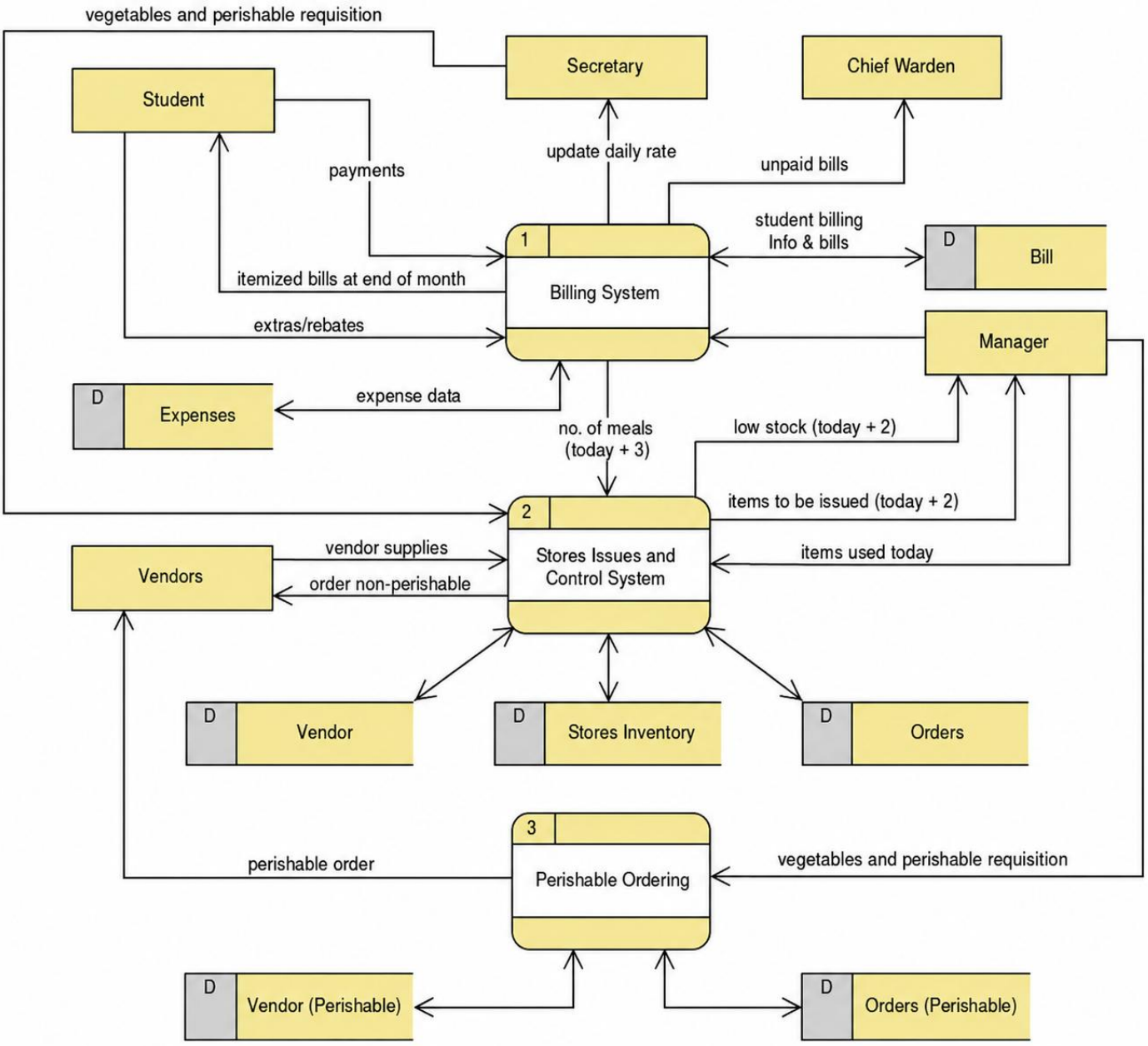
Level 0 DFD

Level 1 DFD

Processes in diagram 0 can be exploded further to represent details of the processing activities. Example below shows the next level ((Diagram 1) of process explosion.

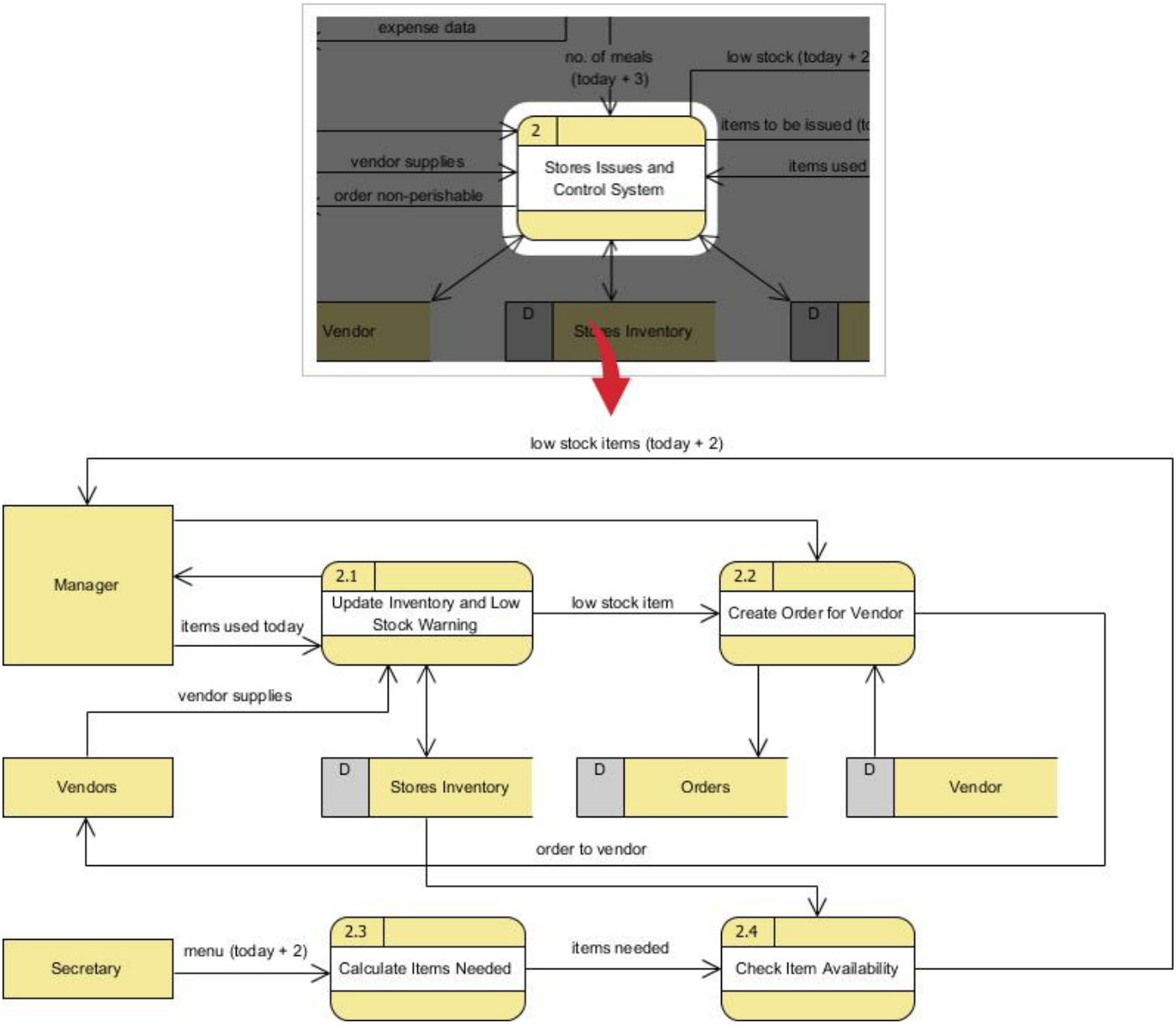
Note that:

Although the following level 1 DFD only has three processes, there are quite a few input and input from the processes to the external entities and that could end up to be a few cross lines among them in the diagram; to avoid this problem, we could use multiple views of the same external entity in the DFD.



Level 2 DFD

When a process in a Level 1 DFD has many data flows with several external entities, its detailed diagram may become difficult to read because of too many connecting or crossing lines. To make the diagram clearer, that particular process can first be separated into a small context-like diagram along with only its related external entities and data flows. After this, the selected process can be further divided into smaller sub-processes in the Level 2 DFD. This approach helps maintain consistency, because all inputs and outputs shown for the process in the Level 1 DFD should also be represented in its detailed Level 2 DFD.



UseCases Diagram

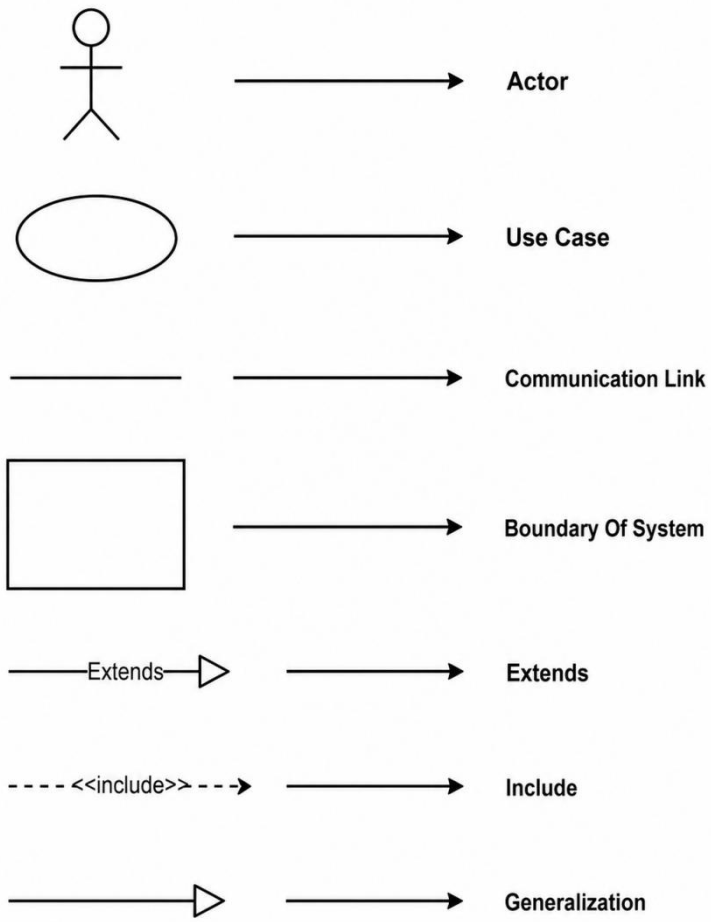
A Use Case Diagram is a UML behavioral diagram that shows how external users or systems, called actors, interact with a system and the services or functionalities, called use cases, that the system provides.

It gives a high-level view of the system from the user's perspective. It is mainly used during requirement analysis to identify what the system should do and who will use it.

It answers:

- ✓ Who uses the system?
- ✓ What services does the system provide?
- ✓ What actions can each user perform?
- ✓ What is inside the system scope and what is outside it?

Notations Used in Use Case Diagrams



1. **Actors:** Actors are external entities that interact with a system to perform specific tasks or achieve goals.

An actor may be:

- ✓ A human user
- ✓ Another software system
- ✓ A hardware device
- ✓ An external organization



Consider an Online Shopping System.

- ✓ Customer is an actor because the customer searches for products and places orders.
- ✓ Admin is an actor because the admin manages products and users.
- ✓ Payment Gateway is an actor because it processes online payments.

2. **Use Cases:** Use Cases represent specific functionalities or services provided by a system to achieve a user's goal. They describe what the system does in response to an actor's interaction.

A use case should normally be written using a verb or action phrase.

Examples

- Login
- Search Product
- Add to Cart
- Make Payment
- Generate Report
- Manage Record

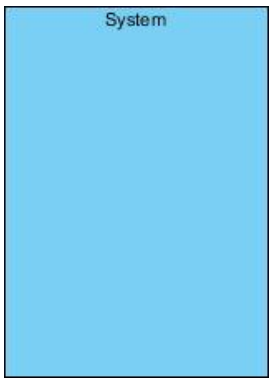


3. **System Boundary:** The System Boundary defines the scope and limits of the system being modeled.

It separates:

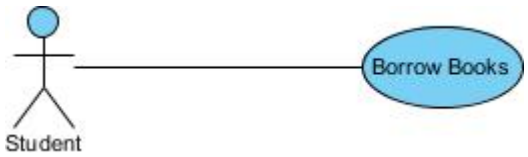
- ✓ Internal system functionalities, which are inside the rectangle
- ✓ External actors, which are outside the rectangle

The system boundary is represented using a rectangle around the use cases.

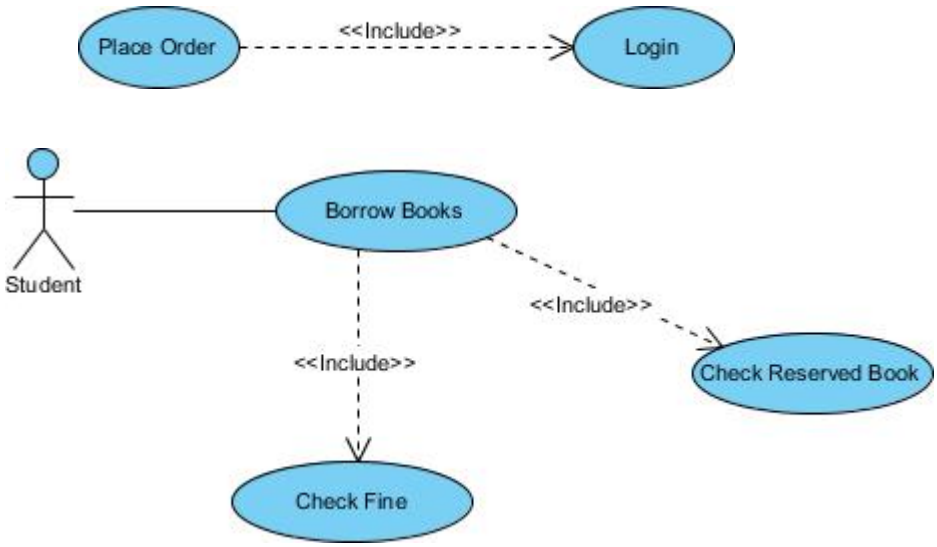


Use Case Diagram Relationships

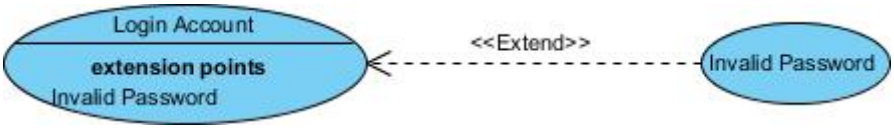
1. **Association Relationship:** The Association Relationship represents a communication or interaction between an actor and a use case. It is shown by a line connecting the actor to the use case.
It means that the actor participates in or uses that functionality described by the use case.

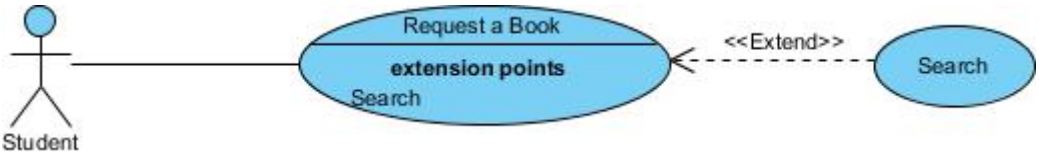


2. **Include Relationship:** The Include Relationship indicates that a use case includes the functionality of another use case. (**mandatory**)
It is represented using a dashed arrow labeled <<include>>. From the main use case → to the included use case

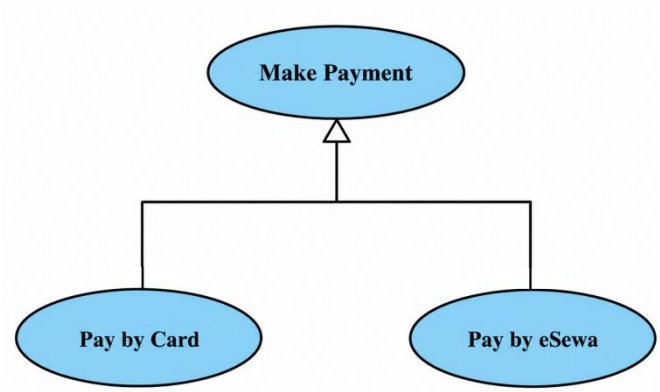


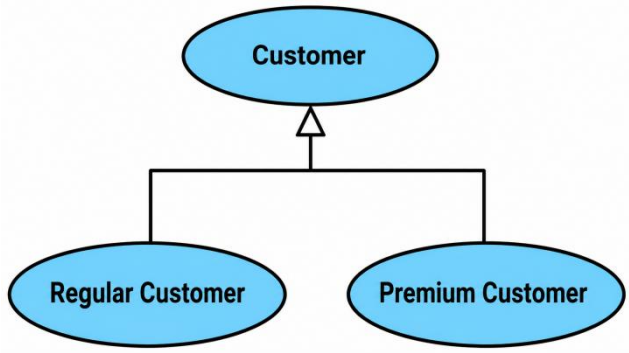
3. **Extend Relationship:** An Extend Relationship shows **optional or conditional** behavior that may be added to a base use case. It is represented by a dashed arrow with the keyword "extend".
From the extending use case → to the base use case.
e.g. Apply Discount Coupon is an optional behavior that may occur while making payment.



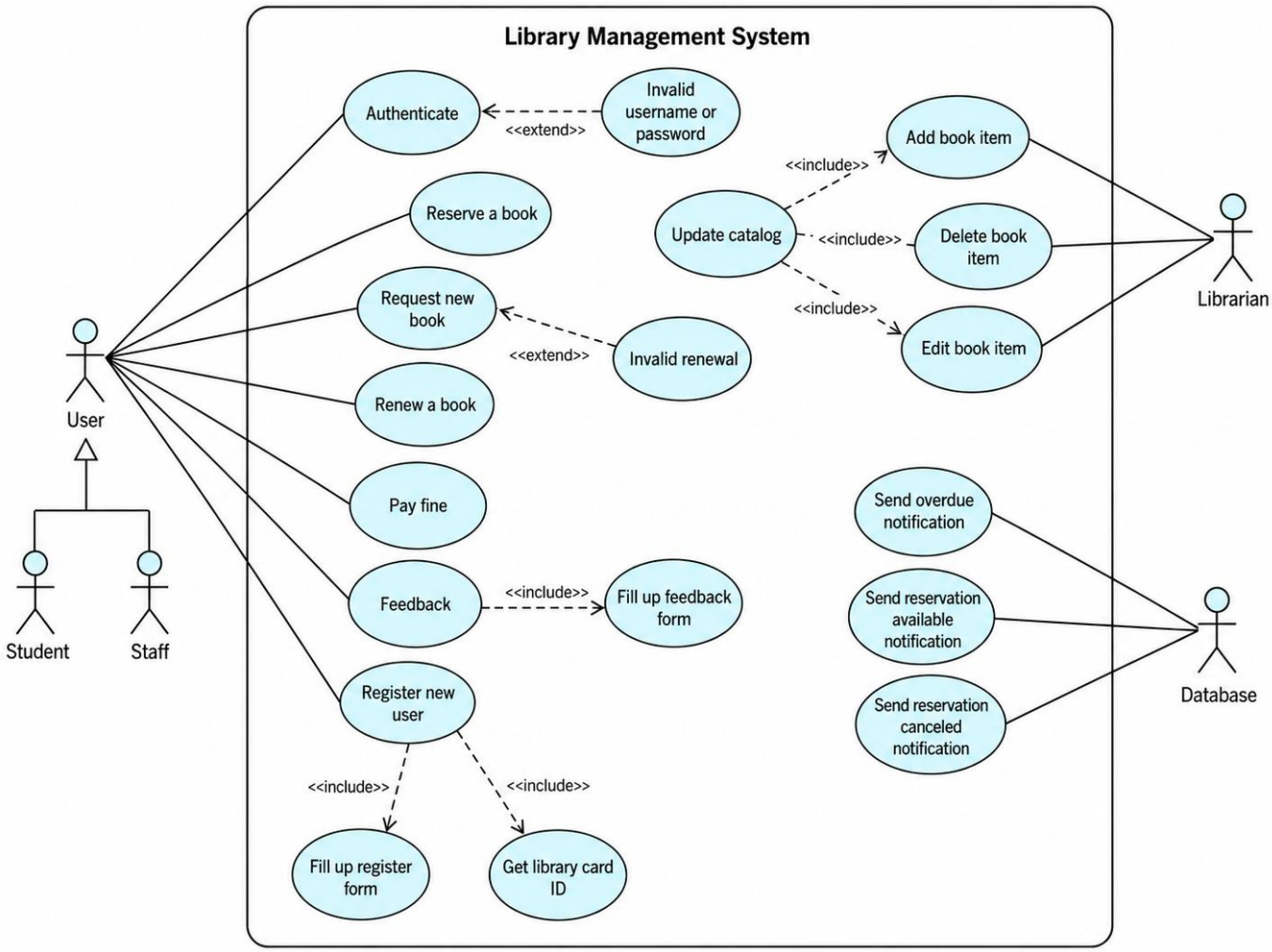


4. **Generalization Relationship:** The Generalization Relationship establishes an "is-a" connection between two use cases, indicating that one use case is a specialized version of another.





Library Management System



Activity Diagram

An Activity Diagram is a UML behavioral diagram used to show the flow of activities, actions, and decisions in a system or process.

It explains how a task is performed step by step, from the beginning to the end.It is **similar to a flowchart, but it follows UML standards** and can also show parallel activities, responsible users/departments, and system interactions.

They answer questions such as:

- ✓ What starts the process?
- ✓ What activities are performed?
- ✓ What decision is made?
- ✓ Which path is followed after a decision?
- ✓ Can two activities happen at the same time?
- ✓ When does the process finish?
- ✓ Which actor or department performs each activity?

For example, in an online shopping system:

- ✓ Customer selects products.
- ✓ Customer places an order.
- ✓ System verifies payment.
- ✓ If payment is successful, the order is confirmed.
- ✓ If payment fails, the customer is asked to try again.

This entire workflow can be represented using an activity diagram.

Notations of an Activity Diagram

1. Initial Node

The Initial Node represents the starting point of the activity diagram. It is shown as a solid black circle.

A diagram normally has one initial node.



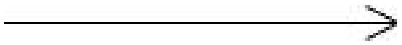
2. Action / Activity

An Action represents a single task or operation performed in the system. It is drawn as a rounded rectangle. Use action names in the form of a verb + object. Because an activity diagram should describe an action, not only an object.



3. Control Flow

A Control Flow shows the direction in which the process moves from one action to another. It is drawn using an arrow.



4. Decision Node

A Decision Node is used when the process has a condition and must choose one path from multiple alternatives. It is represented by a diamond shape.



A decision node normally has:

- ✓ One incoming flow
- ✓ Two or more outgoing flows

The outgoing arrows should contain conditions called guard conditions.

Guard condition examples

- ✓ Yes]
- ✓ [No]
- ✓ [Payment successful]
- ✓ [Payment failed]
- ✓ [Marks ≥ 40]
- ✓ [Seat available]

5. Merge Node

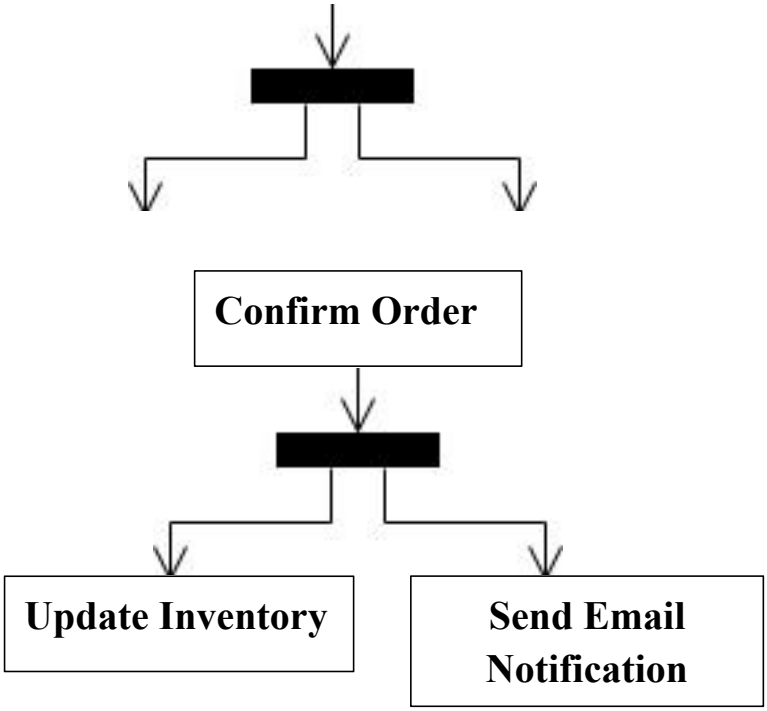
A Merge Node combines alternative paths back into a single flow. It is also represented by a diamond shape.



A merge node is used after a decision when different alternatives later continue to the same next activity.

6. Fork Node

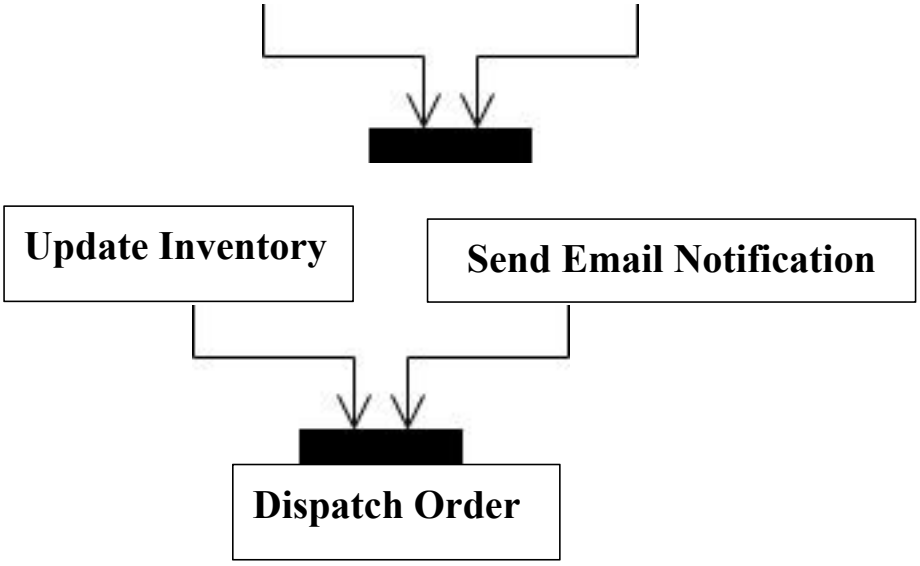
A Fork Node is used when two or more activities need to begin at the same time. It is represented by a thick horizontal or vertical bar.



Both activities can happen simultaneously.

7. Join Node

A Join Node synchronizes parallel activities. It waits until all parallel activities are completed before moving to the next activity.



8. Final Node

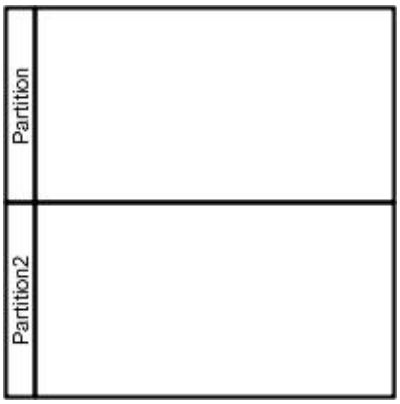
The Final Node represents the end of the whole activity diagram. It is represented by a black circle inside another circle. It means the complete process has ended.

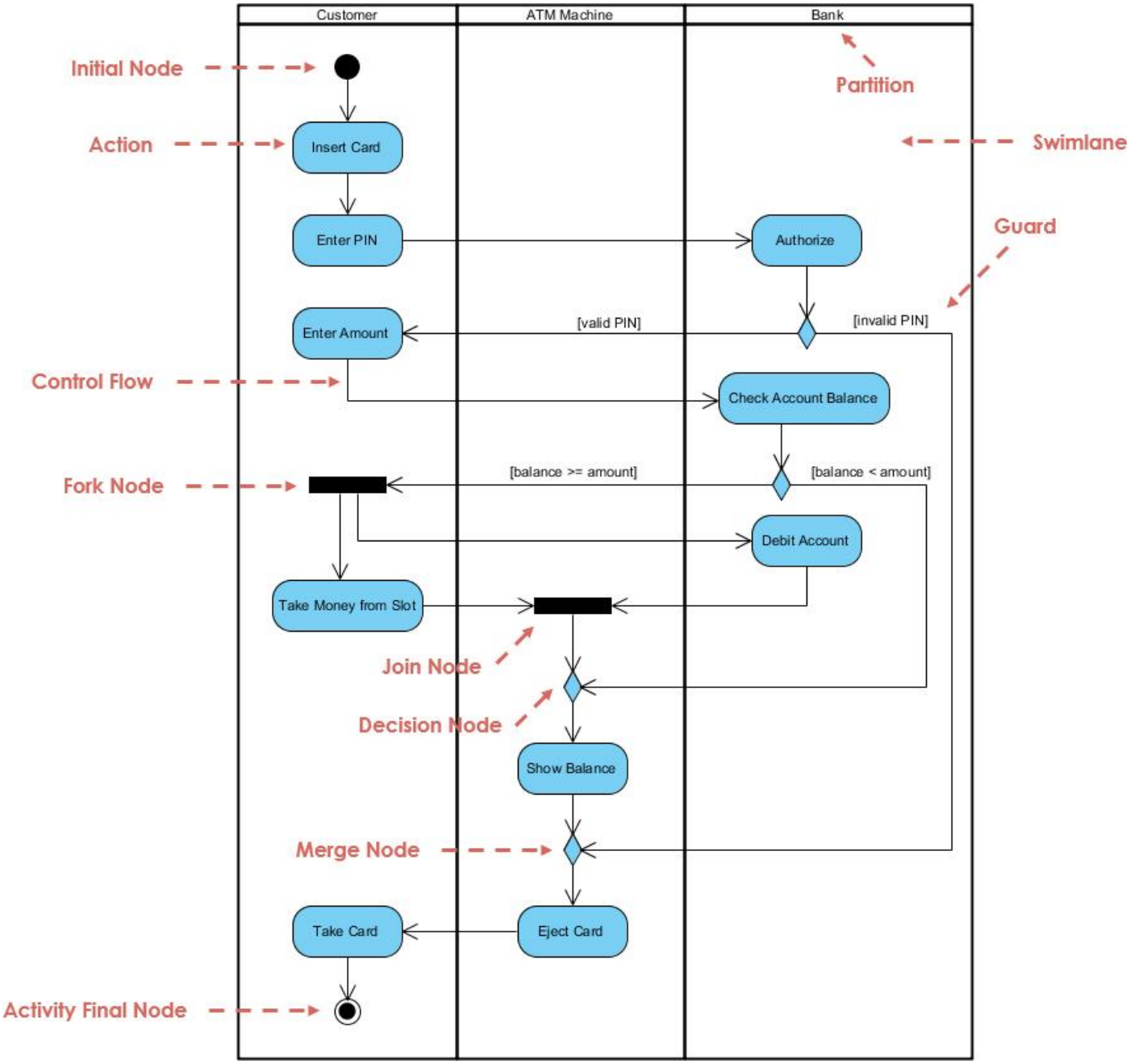


9. Swimlanes / Activity Partitions

Swimlanes divide activities according to the person, department, actor, or system responsible for performing them.

They help show who performs which action.




Customer
ATM Machine
Bank
Initial Node
Partition
Action
Swimlane
Insert Card
Enter PIN
Authorize
Guard
Enter Amount
[valid PIN]
[invalid PIN]
Control Flow
Check Account Balance
Fork Node
[balance >= amount]
[balance < amount]
Debit Account
Take Money from Slot
Join Node
Decision Node
Show Balance
Merge Node
Take Card
Eject Card
Activity Final Node

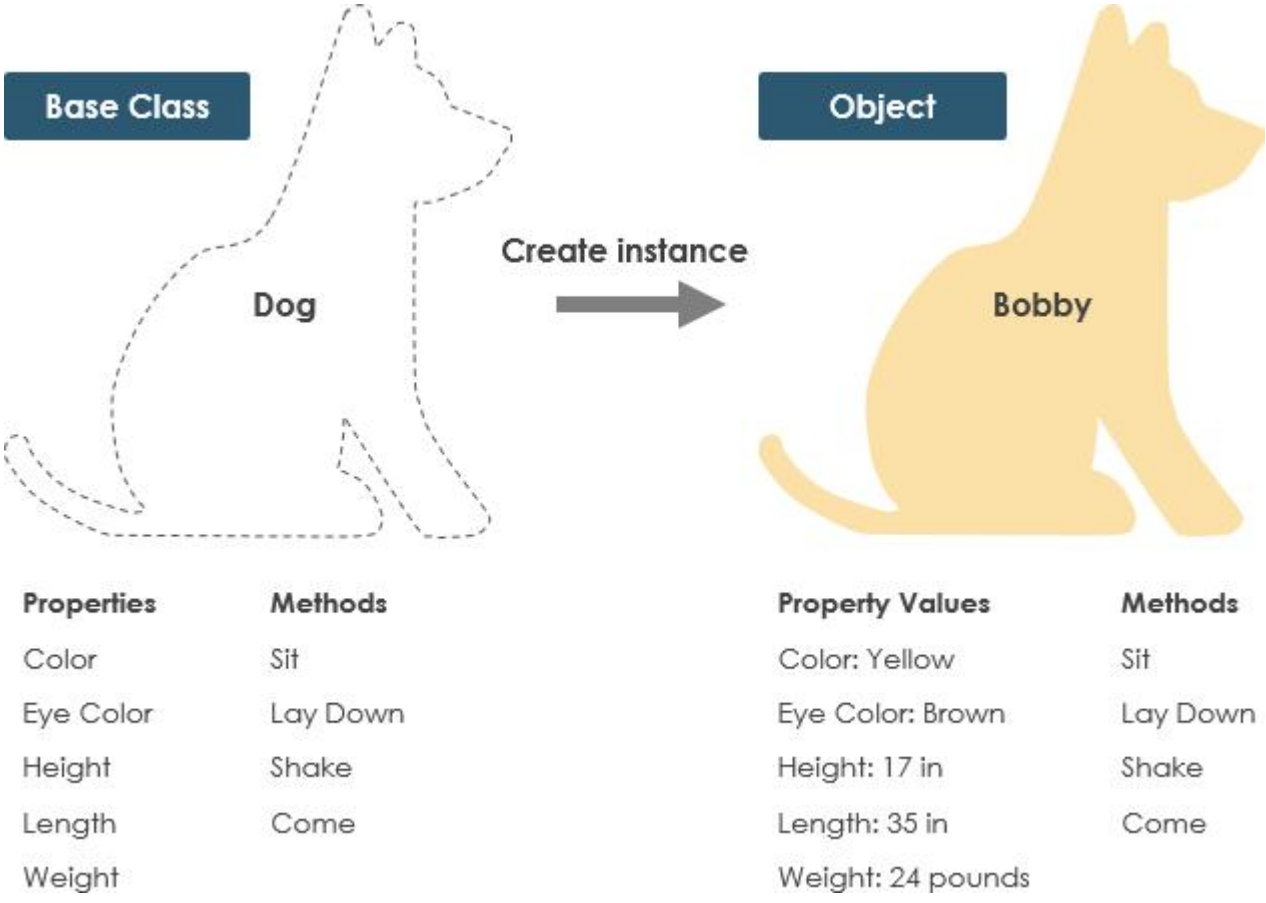
Class Diagram

A UML Class Diagram is a graphical representation used to design and visualize the structure of an object-oriented system. It is one of the most commonly used UML diagrams because it shows the static structure of a system.

A class diagram represents:
Classes – the main entities or objects in the system
Attributes – the data or properties of each class
Operations/Methods – the functions or actions that a class can perform
Relationships – the connections between classes, such as association, inheritance, aggregation, and composition

What is a Class?

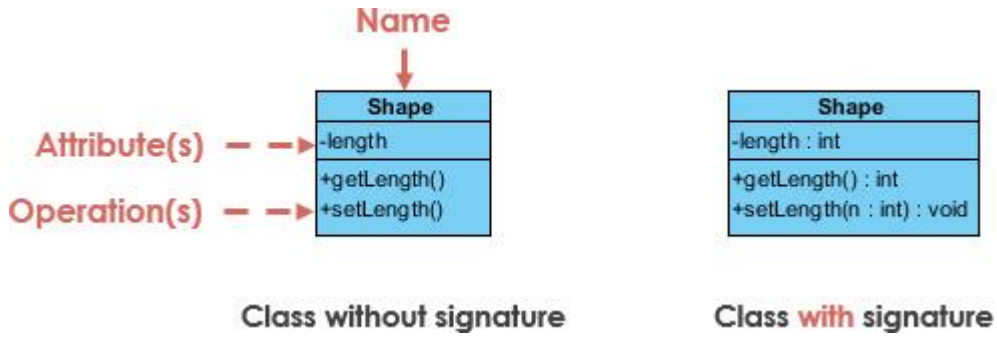
A Class is a blueprint for an object.



Class Notation

A class represent a concept which encapsulates state (attributes) and behavior

(operations). Each attribute has a type. Each operation has a signature. The class name is the only mandatory information.



Class without signature: Here, the attributes and methods are written simply. Their data types, parameters, and return types are not shown.

Class with signature: Here, full details are included:

-length : int

length is an integer attribute.

+getLength() : int

The method returns an integer value.

+setLength(n : int) : void

The method takes one parameter n of type integer and does not return any value.

Class Name:

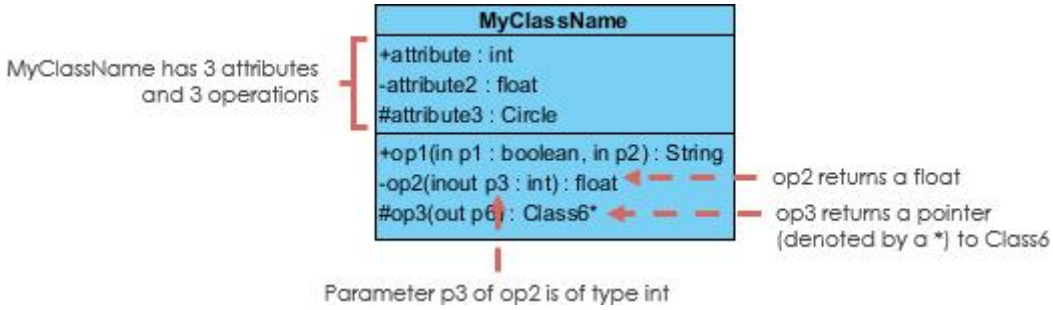
The name of the class appears in the first partition.

Class Attributes:

- ✓ Attributes are shown in the second partition.
- ✓ The attribute type is shown after the colon.
- ✓ Attributes map onto member variables (data members) in code.

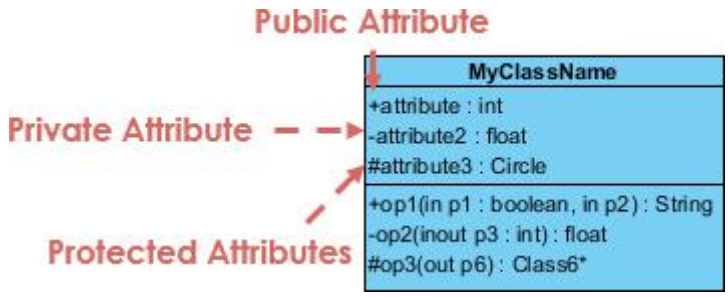
Class Operations (Methods):

- ✓ Operations are shown in the third partition. They are services the class provides.
- ✓ The return type of a method is shown after the colon at the end of the method signature.
- ✓ The return type of method parameters are shown after the colon following the parameter name.



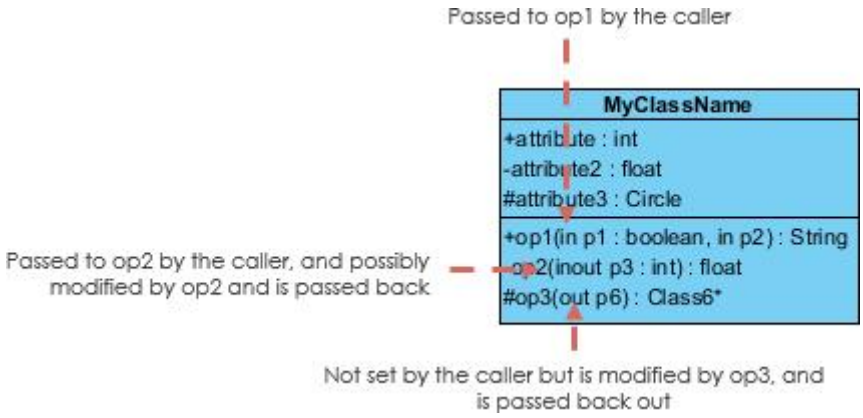
Class Visibility

The +, - and # symbols before an attribute and operation name in a class denote the visibility of the attribute and operation.



Parameter Directionality

Each parameter in an operation (method) may be denoted as in, out or inout which specifies its direction with respect to the caller. This directionality is shown before the parameter name.



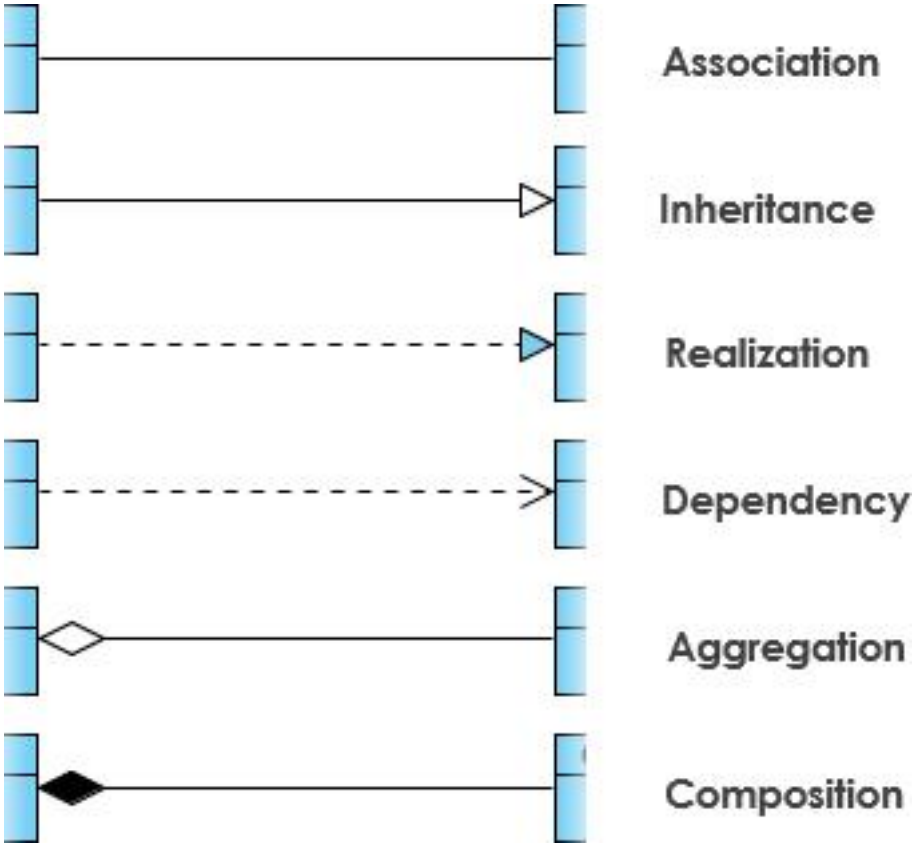
in means the parameter only gives a value to the method. The method uses it as input.
calculateTotal(price, quantity)
Here, price and quantity are input (in) parameters.
Value Movement: Caller → Method

out means the method puts a result into that parameter. It is used to send a value back.
getStudentDetails(out studentName)
The method puts a value such as "Sujan" into studentName.
Value Movement: Method → Caller

inout means the parameter works both ways: a value is sent into the method, and the method may change that same value and send it back.
marks = 50
addBonus(marks)
marks becomes 55
So, marks is used as both input and output.
Value Movement: Caller → Method → Caller

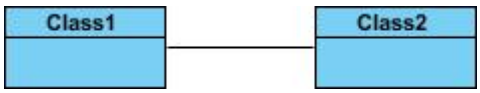
Relationships between classes

UML is not just about pretty pictures. If used correctly, UML precisely conveys how code should be implemented from diagrams. If precisely interpreted, the implemented code will correctly reflect the intent of the designer.



Simple Association:

A structural link between two peer classes.
There is an association between Class1 and Class2.
A solid line connecting two classes.



Cardinality in a class diagram tells us how many objects of one class can be related to objects of another class.

For example, in a relationship between Student and Course, cardinality tells us:

- ✓ How many courses one student can take?
- ✓ How many students can join one course?

The numbers are written near the ends of the relationship line.

1. Exactly one: 1

Student -------- 1 StudentIDCard

Each student has exactly one student ID card.

2. Zero or one: 0..1

Employee -------- 0..1 ParkingSlot

An employee may have no parking slot or only one parking slot. They cannot have two or more parking slots.



3. Zero or more: * or 0..*

Department -------- 0..* Employee

A department may have:

- ✓ no employees yet
- ✓ one employee
- ✓ many employees

So, * means any number, including zero.

4. One or more: 1..*

Order -------- 1..* OrderItem

An order must contain at least one order item, but it can contain many items. It cannot contain zero items.

5. Ordered: {ordered}

Playlist -------- 0..* Song {ordered}

A playlist can contain many songs, and their sequence matters.

For example:

1. Song A

2. Song B

3. Song C

This is different from an unordered collection, where only the songs matter and their position does not.

6. One-to-one, one-to-many, and many-to-many

Person 1 -------- 1 Passport

This is one-to-one.

7. Department 1 -------- 0..* Employee

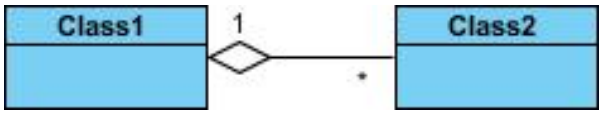
This is one-to-many: one department can have many employees.

8. Student 0..* -------- 0..* Course

This is many-to-many: one student can take many courses, and one course can have many students.

Aggregation:

Aggregation is a special type of association that represents a “whole–part” or “has-a” relationship. Class2 is a part of Class1, and many instances of Class2 may be associated with Class1. However, Class1 and Class2 have separate lifetimes; deleting Class1 does not necessarily delete Class2. In UML, aggregation is represented by solid line with an unfilled diamond placed at the end connected to the whole class.



Example: Department ◇——— Employee

- ✓ Department is the whole.
- ✓ Employee is a part of the department.
- ✓ A department can have many employees.
- ✓ An employee can still exist even if the department is deleted or changed.

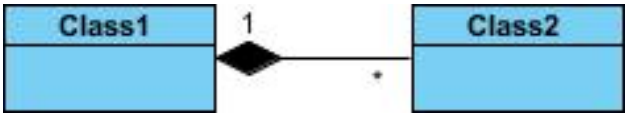
Department 1 ◇——— 0..* Employee

Meaning:

- ✓ One department can have zero or more employees.
- ✓ Each employee is associated with one department.
- ✓ Since this is an aggregation relationship, employees can exist independently even if the department is removed.

Composition :

Composition is a special and strong type of aggregation that represents a whole–part relationship. The objects of Class2 are created, owned, and destroyed with Class1. Class2 cannot exist independently without Class1. In UML, composition is shown by a solid line with a filled diamond placed at the end connected to the composite or whole class.



Example: House ◆——— Room

- ✓ House is the whole.
- ✓ Room is a part of the house.
- ✓ A house may contain many rooms.
- ✓ If the house is destroyed, its rooms are also destroyed.
- ✓ A room cannot normally exist without being part of a house.

House 1 ◆——— 1..* Room

Meaning:

- ✓ One house contains one or more rooms.
- ✓ Each room belongs to exactly one house.

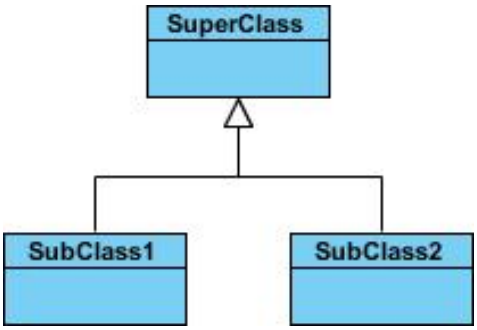
✓ The lifetime of Room depends on the lifetime of House.

Inheritance (or Generalization):

✓ Represents an "is-a" relationship.
✓ An abstract class name is shown in italics. e.g *Vehicle*
✓ It can also be written like this:

<<abstract>>
Vehicle

✓ An abstract class is a parent/base class that is used to define common features for other classes, but you usually do not create an object directly from it.
✓ SubClass1 and SubClass2 are specializations of Super Class.
✓ A solid line with a hollow arrowhead that point from the child to the parent class.



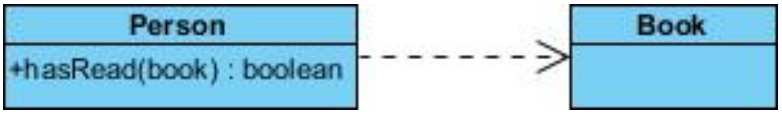
Dependency:

Dependency is a weak relationship in which one class temporarily uses another class. It is shown by a dashed line with an open arrowhead pointing from the dependent class to the class it uses.

For example:

Person - - - - - - - - - > Book

The dashed arrow points from the class that depends on/uses another class toward the class being used. The arrow starts from Person and points to Book. So, Person depends on Book.



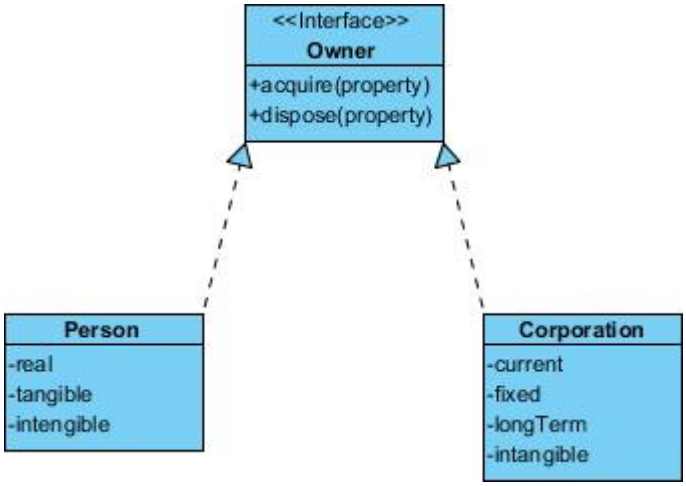
Here, Person uses a Book object only as a parameter of hasRead().
Book is not stored as an attribute inside Person, so this is shown as a dependency, not a normal association.

Meaning

Person depends on Book.
If the definition of Book changes, Person may need to change.
For example, if the Book class changes its method names or properties, the hasRead() method in Person may also need updating.
The reverse is not necessarily true: changing Person does not usually affect Book.

Realization:

Realization represents the relationship between an interface and the class that implements it.



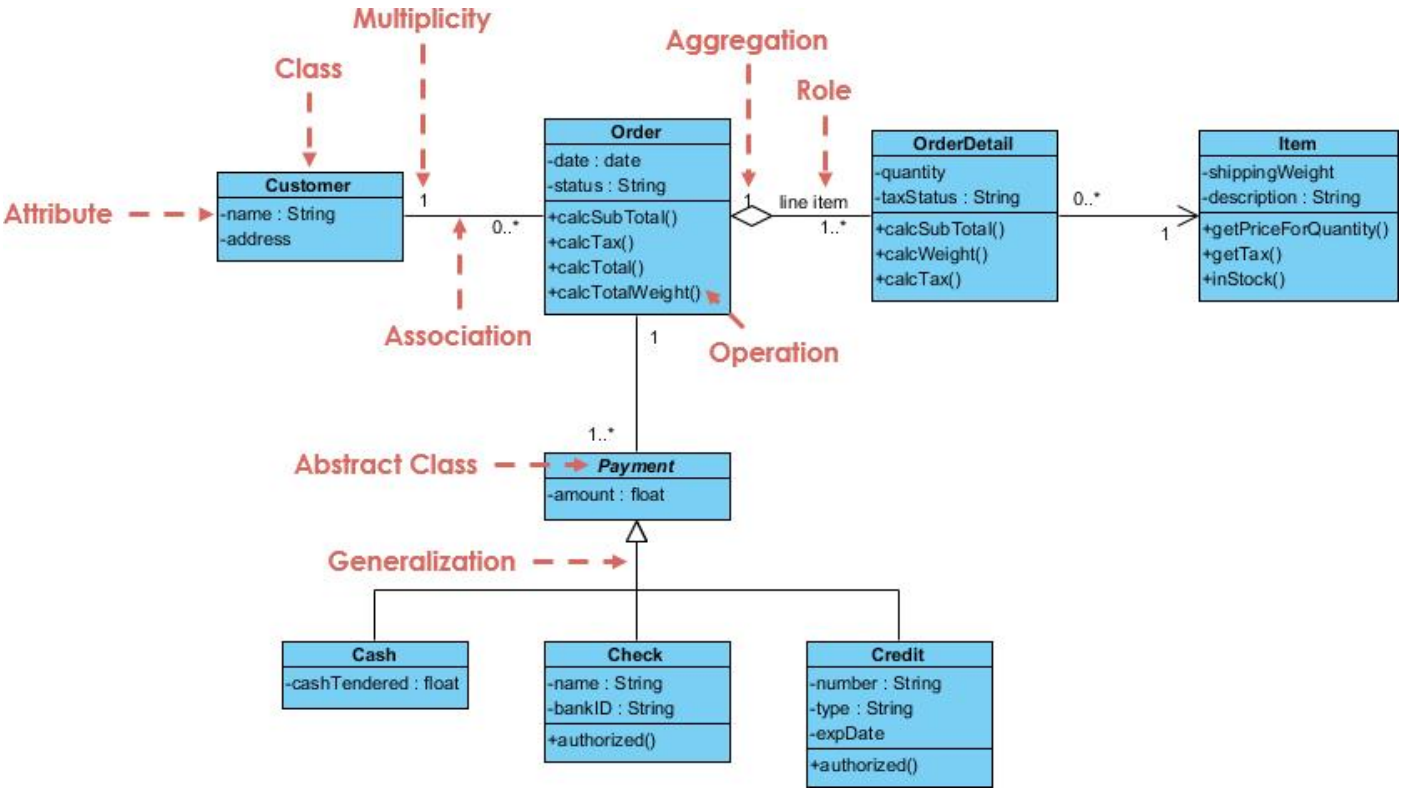
- ✓ Owner is an interface. It only specifies what operations must exist.
- ✓ Person and Corporation implement those operations in their own way.
- ✓ Therefore, Person and Corporation realize the Owner interface.

Relationship Names

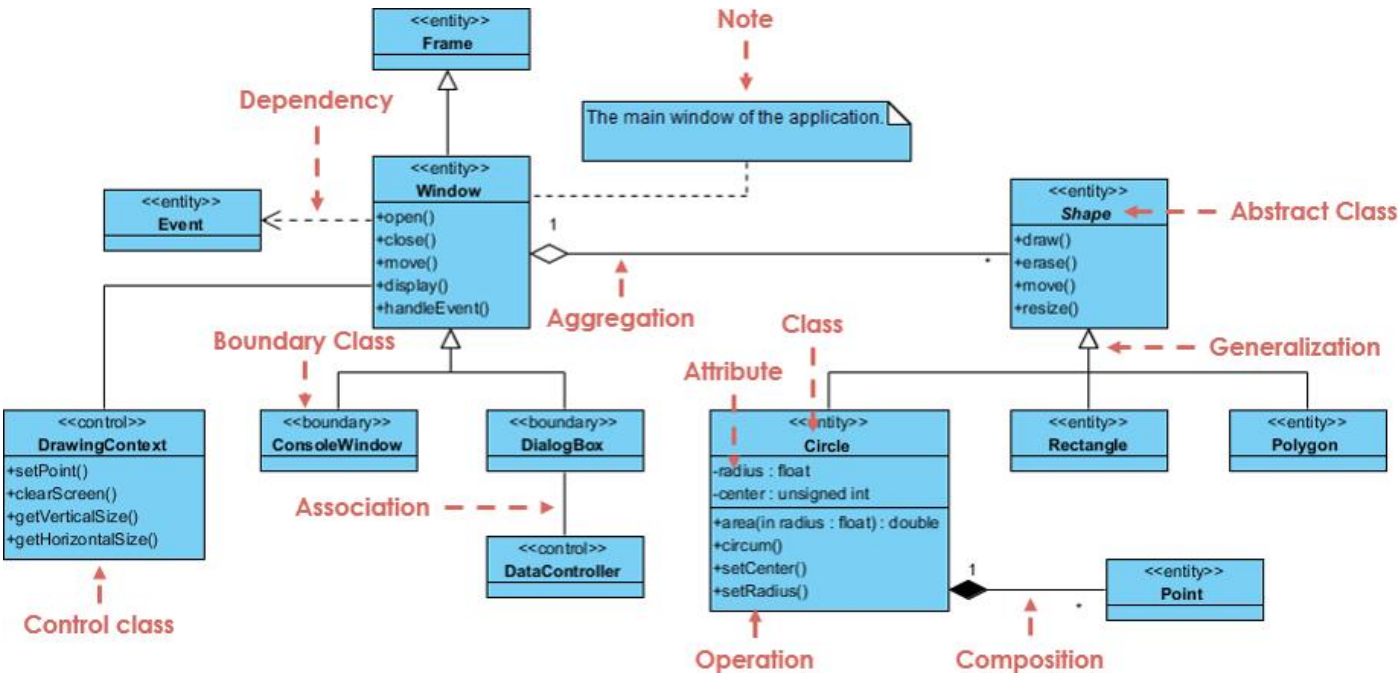
- ✓ Names of relationships are written in the middle of the association line.
- ✓ Good relation names make sense when you read them out loud.



Class Diagram Example: Order System



Class Diagram Example: GUI



Sequence Diagram

A Sequence Diagram is a UML (Unified Modeling Language) behavioral diagram that shows how different objects or participants communicate with one another in a specific sequence over time.

A Sequence Diagram represents the interaction between actors, objects, components, or systems arranged according to the time sequence in which the interactions occur.

The participants are arranged horizontally, while time progresses vertically from top to bottom.

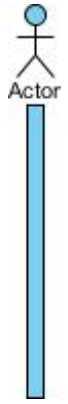
Sequence Diagram Notation

1. Actor

An actor is an external person, organization, device, or system that interacts with the system. An actor is usually shown using a stick figure. An actor normally starts the interaction.

Examples:

- ✓ Customer
- ✓ Administrator
- ✓ Student
- ✓ Teacher
- ✓ Bank Server
- ✓ External Payment System



2. Lifeline

A lifeline is a vertical dashed line drawn below an actor or object. It represents the existence of a participant during the interaction.

The lifeline does not necessarily represent the complete lifetime of an object. It represents the period during which the participant is involved in the particular interaction.

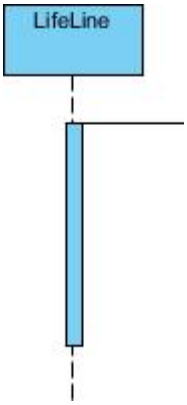


3. Activation Bar

An activation bar, also called an execution specification, is a narrow vertical rectangle drawn on a lifeline.

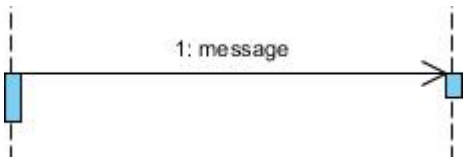
It represents the time during which an object is actively performing an operation. A longer activation bar means the object remains active for a longer period.

Nested activation bars may be used when an object calls another method within its own method.



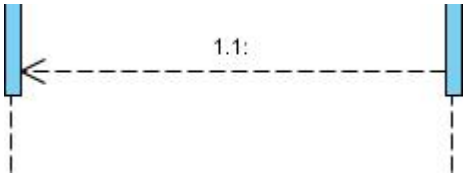
4. Message

A message represents communication between two participants. It is shown using a horizontal arrow from the sender to the receiver.



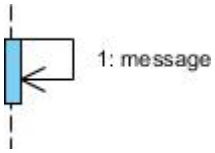
5. Return Message

A return message represents the result returned by the receiver to the sender. It is generally shown using a dashed line with an open arrowhead.



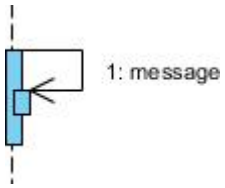
6. Self-Message

A self-message occurs when an object sends a message to itself. It represents a method within an object calling another method of the same object.



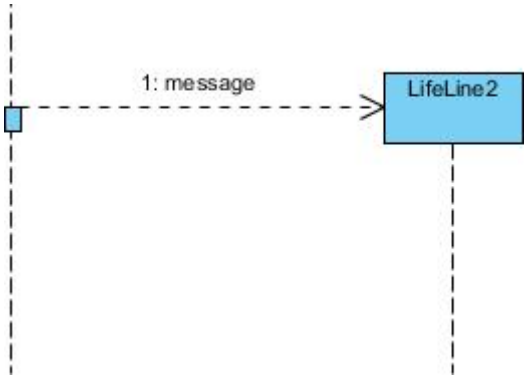
7. Recursive Message

Recursive message is a kind of message that represents the invocation of message of the same lifeline. A recursive message occurs when an object calls the same method again on itself. The object sends a message to itself, and the same operation starts again.



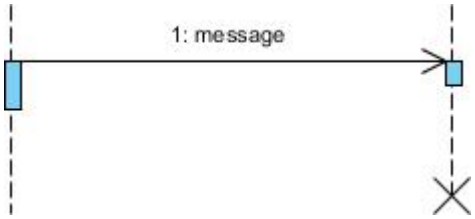
8. Create Message

A create message represents the creation of a new object. The arrow points to the object's rectangle rather than to the top of an existing lifeline. The lifeline of the newly created object starts from the point at which it is created.



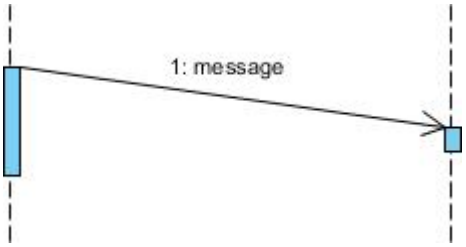
9. Destroy Message

A destroy message represents the destruction or removal of an object. It is shown using a message pointing to an X symbol at the end of the object's lifeline.



10. Duration Message

Duration message shows the distance between two time instants for a message invocation.



11. Found Message

A found message is a message whose sender is unknown or outside the scope of the diagram. It normally starts from a filled circle and points toward a participant.

Example: ● ---- notification() ----> Mobile App

12. Lost Message

A lost message is a message sent to an unknown participant or to a participant outside the scope of the diagram. It normally points toward a filled circle.

Example: System ---- logMessage() ----> ●

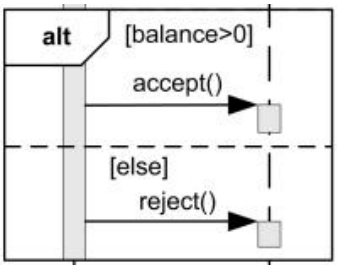
Combined Fragments

Combined fragments are used to represent conditions, alternatives, loops, optional actions, and parallel processing.

A combined fragment is drawn as a rectangular frame surrounding part of the interaction. The operator is written in the upper-left corner.

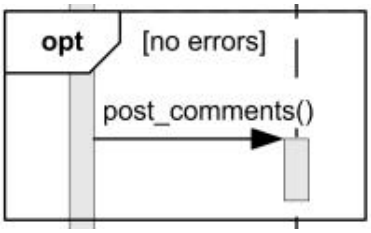
1. alt Fragment

The alt fragment represents alternative flows. It works like an if–else statement. The conditions written inside square brackets are called guard conditions.



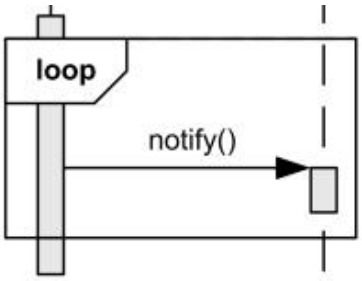
2. opt Fragment

The opt fragment represents optional behavior. It works like an if statement without an else part. The interaction inside the fragment occurs only when the condition is true.

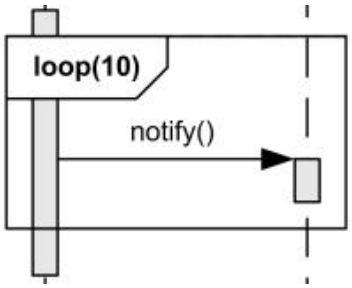


3. loop Fragment

The loop fragment represents repeated interaction. It works like a loop in a programming language.



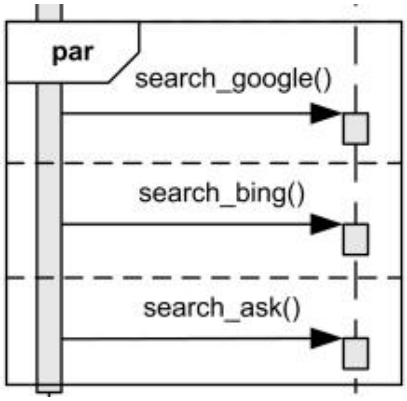
infinite loop



Loop to execute exactly 10 times.

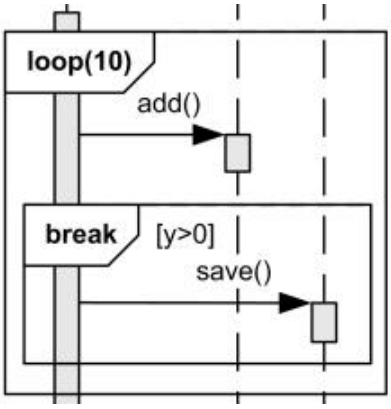
4. par Fragment

The par fragment represents interactions that can occur in parallel. All operands may be executed simultaneously.



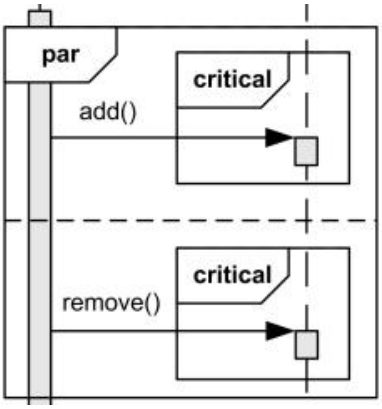
5. break Fragment

The break fragment represents a situation where the normal interaction is stopped when a condition becomes true. The remaining interaction outside the fragment is not executed.



6. critical Fragment

The critical fragment represents a critical section that must be executed without interruption. It is useful when multiple processes are accessing shared resources.
Example:
Read account balance
Deduct amount
Update account balance



7. ref Fragment

The ref fragment refers to another sequence diagram. It is used to avoid drawing the same interaction repeatedly.

Example:

ref Authenticate User

This means that the detailed authentication sequence is provided in another diagram.

8. neg Fragment

The neg fragment represents an invalid or prohibited interaction.

Example:

neg

Unauthenticated user accesses admin page

9. strict Fragment

The strict fragment specifies that the messages must occur exactly in the order shown.

Reference: https://www.uml-diagrams.org/sequence-diagrams-combined-fragment.html#operator-neg

User Register/Login Sequence Diagram

Consider a login system with the following participants:

User

Register Form Controller

Login Form Controller

User Database

